ASSIGNMENT(UNIT-2)

1. Introduction

This assignment is based on Linear Data Structures.

Linear data structures store elements in a sequential manner (one after another).

Examples include:

Array

Linked List

Stack

Queue

In this assignment, we implemented all these data structures using Python from scratch.

2. Dynamic Array

Explanation

A Dynamic Array is an array that automatically increases its size when it becomes full.

- Operations

append(x) → adds an element

pop() → removes the last element

- Working

When the array becomes full → a new array of double size is created

All old elements are copied into the new array

- Time Complexity

Operation Time Complexity

append O(1) (most of the time)

resize O(n)

pop O(1)

- Amortized Analysis ☆

Most append() operations are fast (O(1)),

but sometimes resizing takes O(n) time.

Overall, the average time becomes Amortized O(1).

- Simple meaning:

Since resizing does not happen every time, the average cost remains low.

3. Singly Linked List

Explanation

Each node contains:

data

next pointer (points to next node)

- Operations

Insert at beginning

Insert at end

Delete by value

Traverse (print all elements)

- Time Complexity

Operation Time

Insert at beginning O(1)

Insert at end O(n)

Delete O(n)

Traverse O(n)

4. Doubly Linked List

- Explanation

Each node contains:

data

next pointer

previous pointer

- Operations

Insert after a node

Delete at a position

Time Complexity

- Operation Time

Insert O(n)

Delete O(n)

5. Stack (LIFO)

- Explanation

Stack follows Last In First Out (LIFO).

Example: stack of plates

Operations

push(x) → add element

pop() → remove element

peek() → view top element

- Time Complexity

Operation	Time
push	O(1)
pop	O(1)
peek	O(1)

6. Queue (FIFO)

- Explanation

Queue follows First In First Out (FIFO).

Example: people standing in a line

- Operations

enqueue(x) → add element

dequeue() → remove element

front() → get first element

- Time Complexity

Operation	Time
enqueue	O(1)
dequeue	O(1)
front	O(1)

7. Balanced Parentheses Checker

- Explanation

This problem checks whether brackets are balanced using a Stack.

- Rules

Every opening bracket must match a closing bracket

Order must be correct

- Examples

Input	Output
([])	True
([)]	False
(((	False
""	True

Logic

Push opening brackets into stack

For closing brackets → pop and check match

8. Conclusion

- From this assignment, we learned:

Basics of linear data structures

Difference between arrays and linked lists

Working of stack and queue

Problem solving using stack